Jumpa Verse

SDL2 Game Code Documentation • UML • Libraries • Examples

Jumpa Verse SDL2 Game Documentation

This document explains the current C++ SDL2 game code from basic SDL concepts up to the project structure, object model, rendering pipeline, procedural ground generation, menu system, traps, camera movement, and cleanup. The examples are intentionally simple so a beginner can understand each part before reading the actual project files.

Project: Jumpa-Verse | Main language: C++ | Graphics library: SDL2 | Theme: Orange

By ERROR_343(AWAIS MALIK)

1. Project Overview

Jumpa Verse is an SDL2 side-scrolling platform-style game prototype. SDL2 does not provide a built-in 2D camera, so this project creates the camera illusion by changing object x positions over time. Platforms and traps move left across the fixed window while the player would appear to move through the world.

- main.cpp owns the main game loop, timing, texture loading, font loading, rendering order, and shutdown.
- object.h defines the object structures: hitBox, ground_Class, trapSpike, mainMenuColor, and optionBoxes.
- ground.cpp creates the first platform, generates later random platforms, and removes old ground.
- traps.cpp creates triangular spike traps using SDL2_gfx and frees old trap objects.
- camera.cpp moves ground and trap positions to simulate camera movement.
- menuScreen.cpp renders the menu buttons and handles Play/Exit mouse input.
- Error_Handlers.cpp contains SDL, SDL_image, and SDL_ttf initialization plus cleanup.

2. File Structure

```
Jumpa-Verse/
  main.cpp                  -> program entry point and game loop
  object.h                  -> game classes and object data structures
  functions.h               -> shared function declarations
  Error_Handlers.cpp        -> SDL initialization and shutdown
  ground.cpp                -> random ground generation and ground cleanup
  traps.cpp                 -> spike trap generation and trap cleanup
  camera.cpp                -> camera illusion by moving object positions
  menuScreen.cpp            -> Play and Exit menu
  Images_textures/          -> platform images
  debrosee-font/            -> TTF font used by menu text
  Tutorial_Image/           -> SDL coordinate and triangle reference images
```

3. UML Object Structure

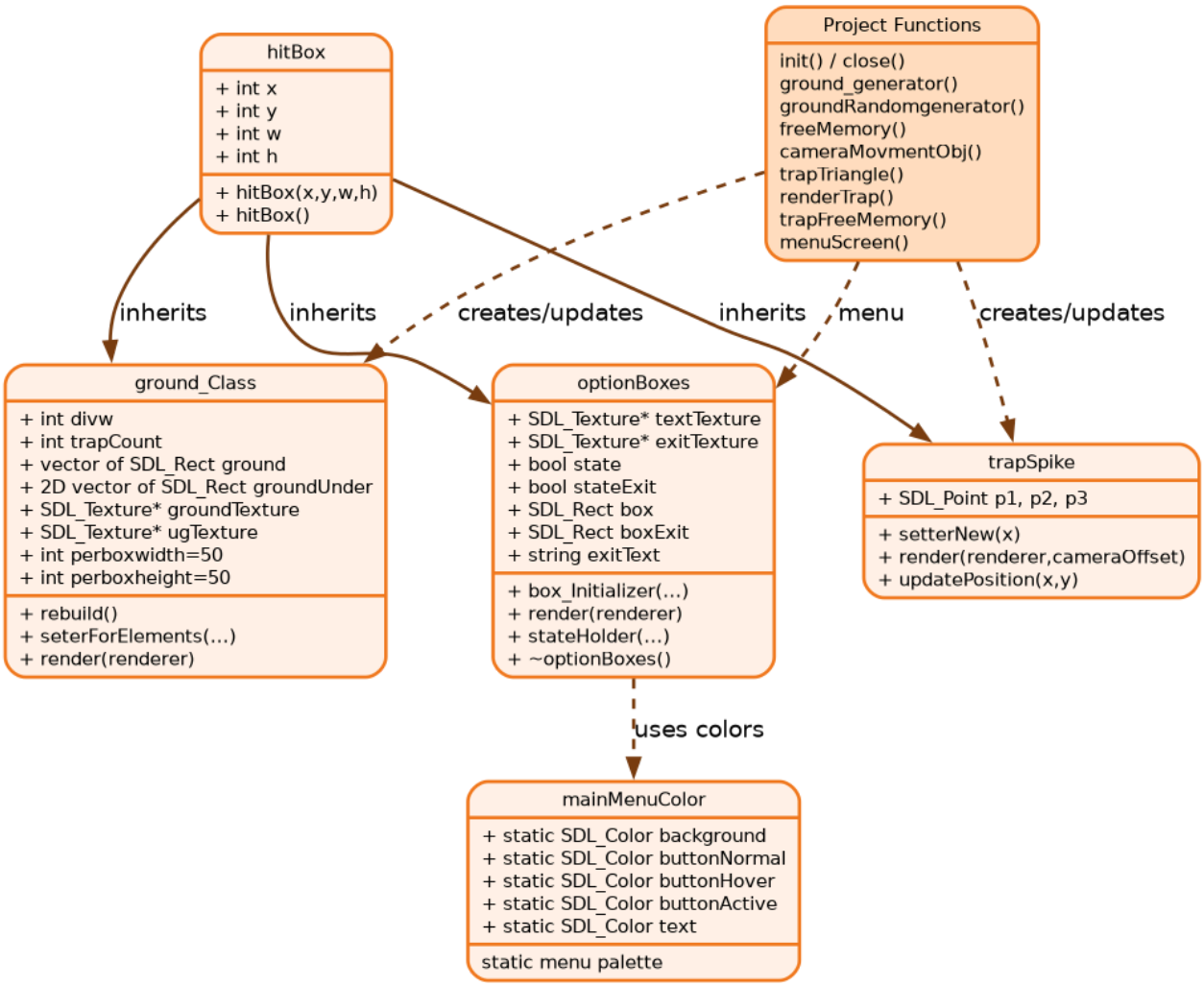

Figure 1: UML class and function relationship diagram

The UML diagram shows that hitBox is the base rectangle structure. Ground platforms, spike traps, and option boxes inherit the same x, y, w, and h fields. This is useful because every visible object needs a position and size.

3.1 Class Details

hitBox

Purpose: stores the rectangle information common to game objects.

```
class hitBox {
public:
    int x, y, w, h;
    hitBox(int x, int y, int w, int h);
    hitBox();
};
```

ground_Class

Purpose: represents one platform. The top is stored in ground, and the underground body is stored in groundUnder. It uses shared textures, meaning the class does not load or own the image files itself.

- divw: number of 50-pixel columns across the platform width.
- trapCount: number of traps currently created on that platform.
- ground: one-dimensional vector of SDL_Rect top tiles.
- groundUnder: two-dimensional vector of SDL_Rect lower tiles.
- rebuild(): recalculates every tile rectangle when x/y/w/h changes.
- seterForElements(): changes platform bounds and rebuilds geometry.
- render(): draws the top and underground tiles with SDL_RenderCopy.

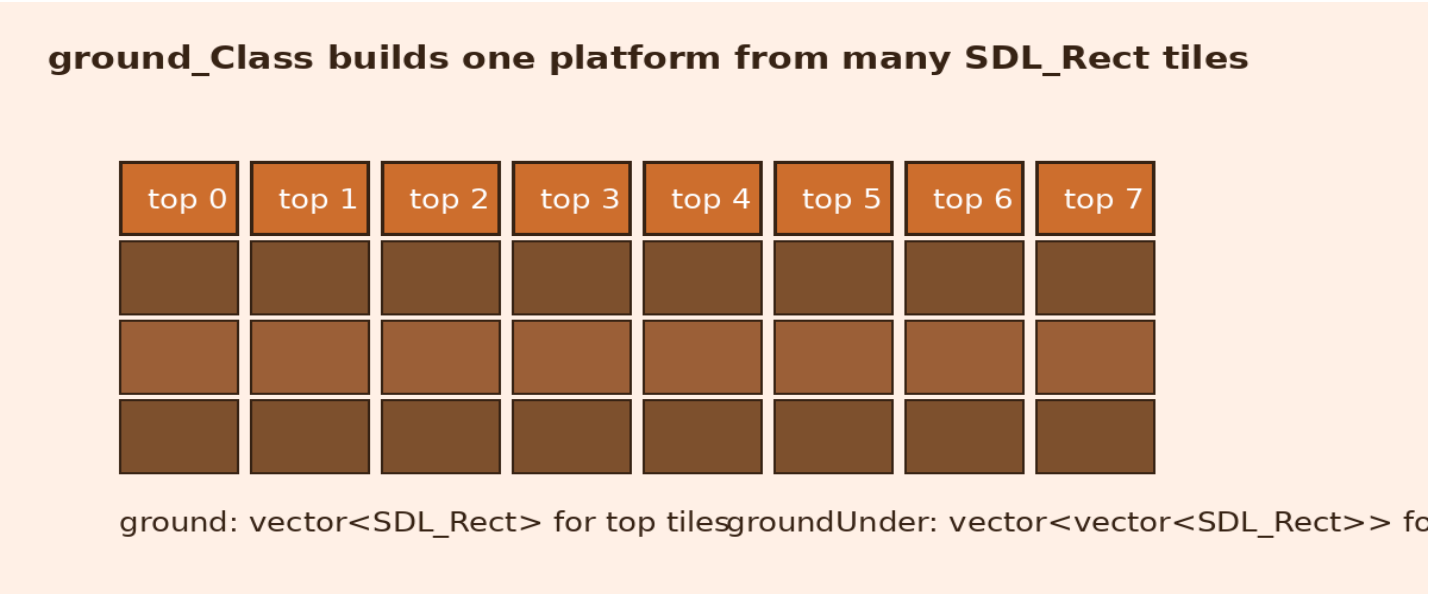

Figure 2: How ground_Class breaks one platform into SDL_Rect tiles

trapSpike

Purpose: represents one triangular spike. It inherits the hitBox rectangle but draws itself using three SDL_Point values. SDL2_gfx functions filledTrigonRGBA and trigonRGBA draw the filled triangle and outline.

```
trapSpike(int x, int y, int w, int h) : hitBox(x, y, w, h) {
    p1 = {x, y};
    p2 = {x + 50, y};
    p3 = {x + 25, y - 50};
}
```

optionBoxes

Purpose: stores and renders the Play and Exit menu buttons. It uses SDL_ttf to convert button text into textures. The destructor frees textTexture and exitTexture, which is important because SDL textures use memory outside ordinary C++ objects.

4. Runtime Flow

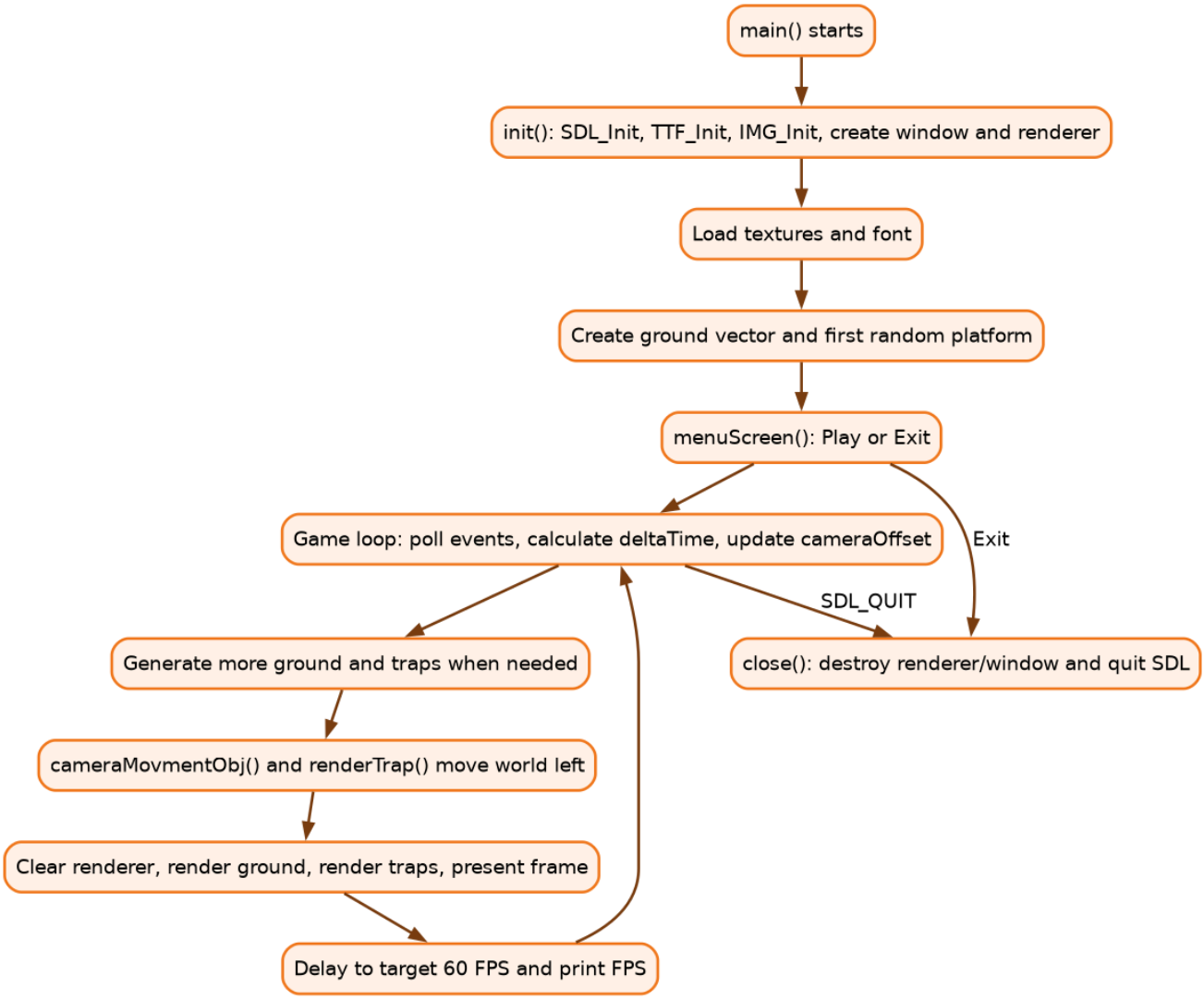

Figure 3: Program flow from initialization to game loop and cleanup

The game starts by creating SDL systems, then loading textures and fonts. The menu runs before the world update. After Play is clicked, each frame calculates deltaTime, updates the camera offset, creates new platforms if needed, deletes old objects, renders the frame, and delays to keep the frame rate near 60 FPS.

5. SDL2 Basics

SDL2 is a cross-platform C library used from C++ to create windows, read input, draw graphics, play audio, and manage timing. It is low-level compared with a full game engine: SDL gives you the building blocks, while your code decides the game structure.

5.1 Minimal SDL Program

```
#include <SDL2/SDL.h>

int main() {
    SDL_Init(SDL_INIT_VIDEO);
    SDL_Window* window = SDL_CreateWindow(
        "Example", SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED,
```

```
        800, 600, SDL_WINDOW_SHOWN
    );
    SDL_Renderer* renderer = SDL_CreateRenderer(window, -1, SDL_RENDERER_ACCELERATED);

    SDL_SetRenderDrawColor(renderer, 255, 128, 0, 255);
    SDL_RenderClear(renderer);
    SDL_RenderPresent(renderer);
    SDL_Delay(2000);

    SDL_DestroyRenderer(renderer);
    SDL_DestroyWindow(window);
    SDL_Quit();
}
```

Important idea: the renderer draws into a hidden back buffer. SDL_RenderPresent swaps that finished frame onto the screen.

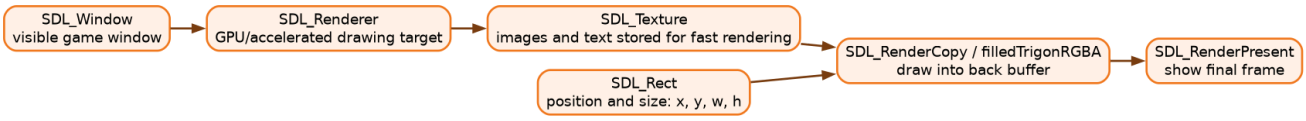

Figure 4: SDL rendering pipeline used by the project

5.2 SDL Coordinates and Rectangles

SDL uses a screen coordinate system where x increases to the right and y increases downward. The top-left corner is usually (0, 0). An SDL_Rect stores x, y, width, and height.

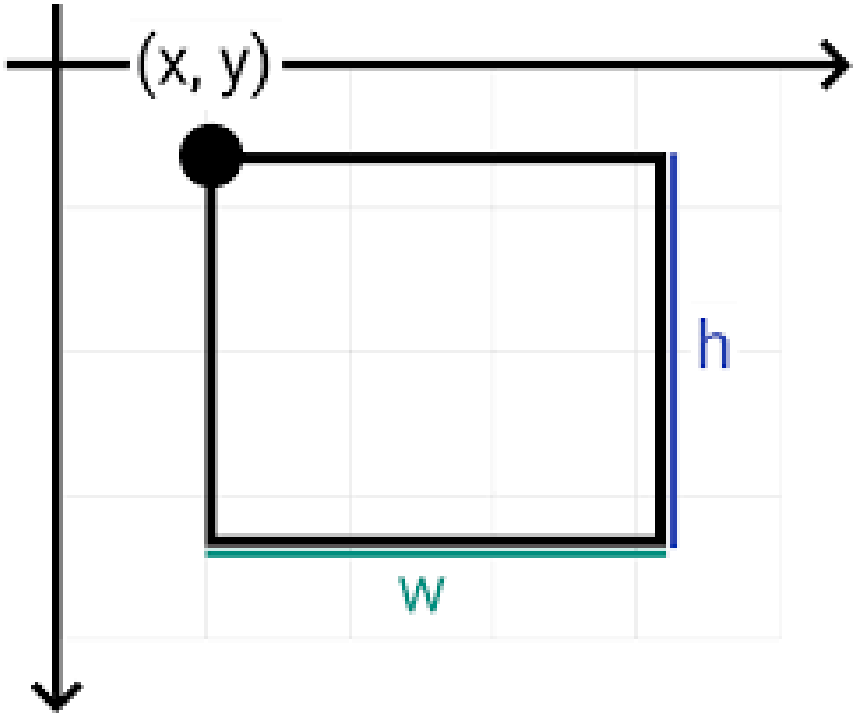

Figure 5: SDL2 coordinate reference from the project tutorial images

```
SDL_Rect box;
box.x = 100;  // left position
box.y = 200;  // top position
box.w = 80;   // width
box.h = 50;   // height
```

```
SDL_SetRenderDrawColor(renderer, 255, 128, 0, 255);
SDL_RenderFillRect(renderer, &box);
```

5.3 Events and Input

SDL_Event stores input and window messages. SDL_PollEvent reads all waiting events. This project uses SDL_QUIT to close the game and SDL_MOUSEBUTTONDOWN to detect menu clicks.

```
SDL_Event event;
while (SDL_PollEvent(&event)) {
    if (event.type == SDL_QUIT) {
        quit = true;
    }
    if (event.type == SDL_MOUSEBUTTONDOWN) {
        int mx = event.button.x;
        int my = event.button.y;
        // compare mx/my with a button rectangle
    }
}
```

5.4 Timing and Delta Time

deltaTime is the number of seconds since the previous frame. Movement becomes smoother because objects move according to elapsed time instead of assuming every computer runs the same number of frames per second.

```
Uint32 currentTime = SDL_GetTicks();
float deltaTime = (currentTime - lastTime) / 1000.0f;
lastTime = currentTime;

float speed = 2.0f;
cameraOffset -= speed * deltaTime;
```

6. SDL_image

SDL_image loads image files that plain SDL cannot load easily. In this project it is used for JPEG platform textures.

```
if (!(IMG_Init(IMG_INIT_JPG) & IMG_INIT_JPG)) {
    std::cerr << IMG_GetError() << std::endl;
}

SDL_Texture* groundTex = IMG_LoadTexture(renderer, "Images_textures/ground.jpeg");
SDL_Texture* ugTex = IMG_LoadTexture(renderer, "Images_textures/underGround.jpeg");

SDL_RenderCopy(renderer, groundTex, nullptr, &destinationRect);
```

Best practice: load each texture once, reuse it many times, and destroy it when the program ends. The current ground_Class stores texture pointers as shared references.

7. SDL_ttf

SDL_ttf loads TrueType fonts and creates text surfaces. The surface is then converted into an SDL_Texture for fast rendering.

```
TTF_Init();
TTF_Font* font = TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf", 100);

SDL_Color orange = {255, 127, 0, 255};
SDL_Surface* surface = TTF_RenderText_Blended(font, "Play", orange);
SDL_Texture* textTexture = SDL_CreateTextureFromSurface(renderer, surface);
```

```
SDL_FreeSurface(surface);

SDL_RenderCopy(renderer, textTexture, nullptr, &textRect);
SDL_DestroyTexture(textTexture);
TTF_CloseFont(font);
TTF_Quit();
```

In menuScreen.cpp, optionBoxes uses this same technique for the Play and Exit labels. Hovering changes state, then the text texture is recreated with a different color.

8. SDL2_gfx

SDL2_gfx adds drawing primitives that SDL2 does not provide directly, such as filled triangles, circles, polygons, and anti-aliased lines. The project uses it for spike traps.

```
SDL2_GFXPRIMITIVES_SCOPE int trigonRGBA ( SDL_Renderer *  renderer,
                                          Sint16          x1,
                                          Sint16          y1,
                                          Sint16          x2,
                                          Sint16          y2,
                                          Sint16          x3,
                                          Sint16          y3,
                                          Uint8           r,
                                          Uint8           g,
                                          Uint8           b,
                                          Uint8           a
                                        )
```

Figure 6: Triangle color and point reference from the project tutorial images

```
#include <SDL2/SDL2_gfxPrimitives.h>

filledTrigonRGBA(renderer,
    p1.x, p1.y,
    p2.x, p2.y,
    p3.x, p3.y,
    92, 52, 40, 255);

trigonRGBA(renderer,
    p1.x, p1.y,
    p2.x, p2.y,
    p3.x, p3.y,
    40, 22, 18, 255);
```

The first call fills the triangle. The second call draws the outline. RGBA means red, green, blue, and alpha transparency.

9. Game Camera and World Movement

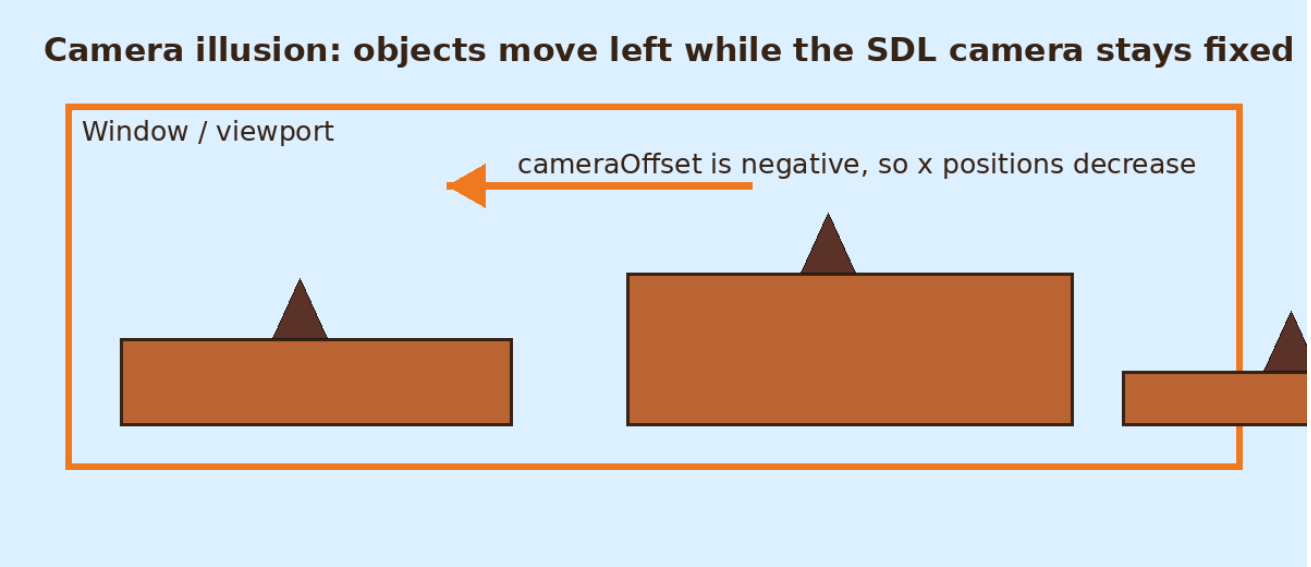

Figure 7: The camera illusion used by Jumpa Verse

The project keeps the SDL window fixed. Instead of moving an actual camera, the code moves ground and trap objects by changing their x positions. When x gets smaller, the object appears to travel left.

```
void cameraMovmentObj(std::vector<ground_Class>& ground, float cameraOffset,
                      std::vector<trapSpike>& trap) {
    for (int i = 0; i < ground.size(); i++) {
        ground[i].seterForElements(
            ground[i].x + (int)cameraOffset,
            ground[i].y,
            ground[i].w,
            ground[i].h
        );
    }
}
```

10. Procedural Ground and Traps

ground_generator creates the first platform. groundRandomgenerator creates later platforms using random x, y, width, and height values based on the last platform. This makes the level continue without manually placing every platform.

```
std::uniform_int_distribution<> distx(
    prevGround.back().x + prevGround.back().w + 120,
    prevGround.back().x + prevGround.back().w + 200
);

std::uniform_int_distribution<> disty(
    prevGround.back().y - 100,
    prevGround.back().y + 600
);
```

trapTriangle may place a trap on a platform if a random chance succeeds and that platform has fewer than four traps. trapFreeMemory limits the trap vector so old traps do not grow forever.

11. Memory and Cleanup

- SDL_DestroyRenderer(renderer) frees the renderer.
- SDL_DestroyWindow(window) frees the window.
- SDL_DestroyTexture(texture) should be called for textures when no longer needed.
- SDL_FreeSurface(surface) is required after converting a text surface into a texture.
- IMG_Quit(), TTF_Quit(), and SDL_Quit() shut down SDL libraries.
- Vectors such as ground and trap should erase objects that are off-screen to avoid unnecessary memory use.

```
void close(SDL_Window* window, SDL_Renderer* renderer) {
    SDL_DestroyRenderer(renderer);
    SDL_DestroyWindow(window);
    IMG_Quit();
    SDL_Quit();
}
```

Recommended improvement for this project: close() should also call TTF_Quit(), and main.cpp should destroy groundTex and ugTex after the game loop.

12. Beginner-Friendly Build Notes

A typical Linux compile command for this project should link SDL2, SDL2_image, SDL2_ttf, and SDL2_gfx.

```
g++ main.cpp Error_Handlers.cpp ground.cpp camera.cpp traps.cpp menuScreen.cpp \
    -o test \
    -lSDL2 -lSDL2_image -lSDL2_ttf -lSDL2_gfx
```

If headers are missing, install the development packages for SDL2, SDL2_image, SDL2_ttf, and SDL2_gfx for your Linux distribution.

13. How To Read The Code

- Start from main.cpp because it shows the complete lifecycle.
- Read init() next to understand SDL setup.
- Read object.h to understand what data each object stores.
- Read ground.cpp and traps.cpp to understand level generation.
- Read camera.cpp to understand why objects move left.
- Read menuScreen.cpp to understand SDL mouse input and SDL_ttf text rendering.

14. Suggested Improvements

- Add a player class with position, velocity, jump physics, collision checks, and animation state.
- Destroy loaded textures and close the font before exiting.
- Move constants such as window size, tile size, and colors into a config header.
- Use size_t for vector indexes where possible.
- Separate rendering, updating, and input into clearer functions as the game grows.

- Add collision detection between player hitBox and trapSpike/ground_Class.

15. Quick Glossary

- Window: the operating-system window where the game appears.
- Renderer: SDL object used to draw images, rectangles, and primitives.
- Texture: GPU-friendly image used for fast rendering.
- Surface: CPU-side image data, often temporary before creating a texture.
- Rect: x/y/w/h rectangle used for position and size.
- Event: keyboard, mouse, quit, and other input message.
- Delta time: elapsed time between frames.
- RGBA: red, green, blue, alpha color values.